



University of Science, VNU - HCM

Lab 03: 2D object Coloring with OpenGL

Course: **Computer Grapics**

Course ID: **CS411**

Name: **Le Tien Dat**

Student ID: **23125028**

Class: **23APCS2**

Semester 2, 2025 - 2026

March 11, 2026.

Contents

1	Objective	2
2	Features	2
2.1	Interactive Polygon Drawing	2
2.2	Contextual Shape Spawning	3
2.3	Hovering & Dynamic Menus	3
2.4	Affine Transformations	4
3	Technical Implementation & Algorithms	5
3.1	Ray-Casting Algorithm (Hit Detection)	5
3.2	OpenGL Matrix Pipeline	6
3.3	FreeGLUT Menu State Management	6
4	Conclusion	7

1 Objective

The objective of this project is to implement an interactive 2D graphics application using C++, OpenGL, and the FreeGLUT library. The software focuses on geometry generation, user interaction mapping, shape coordinate bounds detection, and fully featured 2D Affine Transformations securely applied via matrix stacks.

2 Features

The application operates on a strict event-driven UI capable of handling the following mechanics, directly fulfilling the lab requirements:

2.1 Interactive Polygon Drawing

- **Left-Click:** Adds a geometric vertex dynamically. As points are added, a live preview is drawn in red (`GL_LINE_STRIP` and `GL_POINTS`) indicating the unfinished construction.
- **Right-Click:** Evaluates the vertices, resolves the final connection to close the geometric loop, calculates the shape's relative center (cx , cy), and renders the final closed polygon to the screen.
- **Overlap Prevention:** All clicks are rigorously evaluated. The program mathematically blocks users from placing a new starting vertex or any subsequent vertices explicitly *inside* an existing shape's bounds.

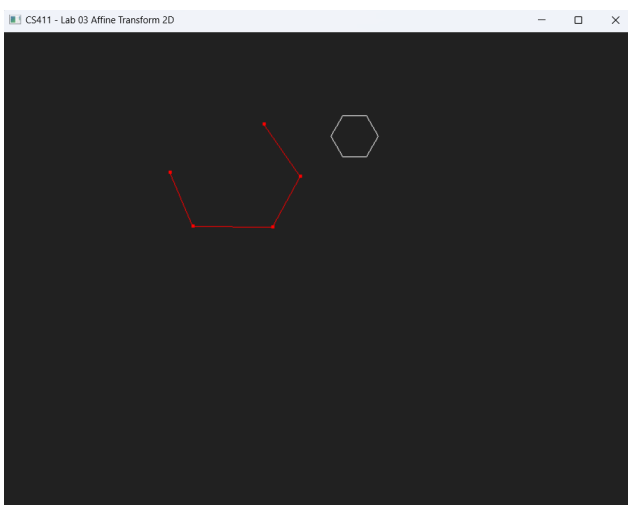


Figure 1: Preview when draw shape

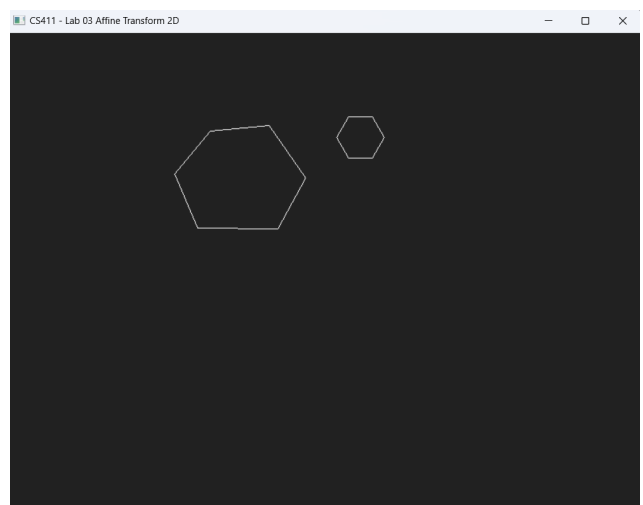


Figure 2: Done drawing interactive

2.2 Contextual Shape Spawning

If the user right-clicks on the blank viewport void, a context menu is generated listing predefined geometric templates (e.g., Triangle, Rectangle, Hexagon, Ellipse, Plus, Minus, Star), which has the implementation from the previous lab. These templates adapt their localized generation vectors strictly mapped to the exact coordinates (`spawnX`, `spawnY`) of the user's triggering right-click cursor location.

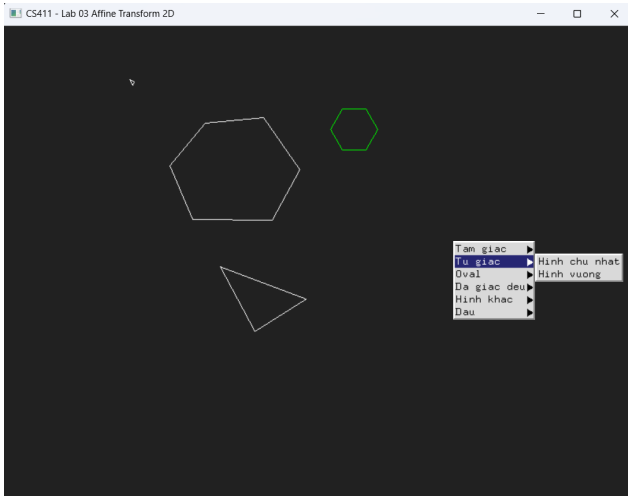


Figure 3: Context Menu to draw shape

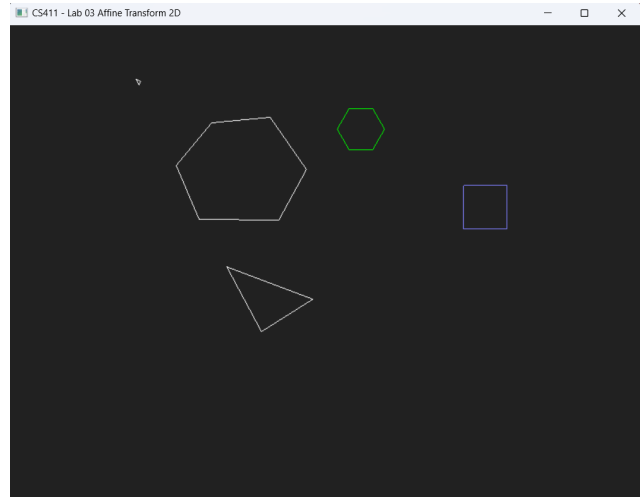


Figure 4: A shape is correctly created

2.3 Hovering & Dynamic Menus

- **Color Indication:**
 - **White:** Default shape state.
 - **Blue (Hovered):** Triggered when the system tracks `PassiveMotion` coordinates inside the boundaries of a shape segment.
 - **Green (Selected):** Fully targeted shape enabled under the "Select this shape" menu lock.
- **Context-Switching UI:** The `GLUT_RIGHT_BUTTON` behaves dynamically depending strictly on the hover state evaluated before the trigger. Right-clicking a void opens the "Shape Catalog Menu", while right-clicking a shape opens the "Select this shape" evaluation menu.

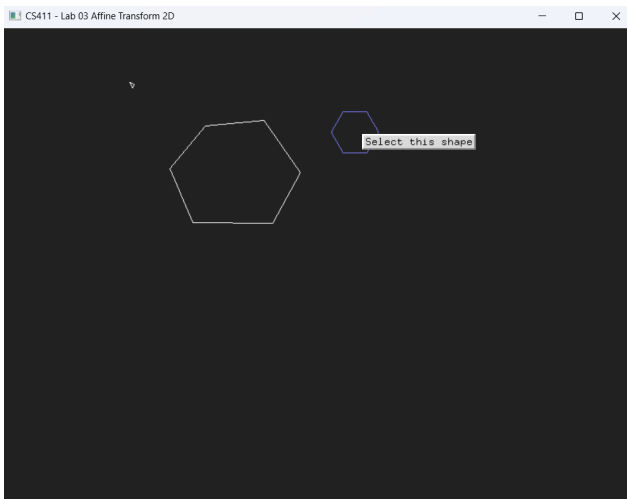


Figure 5: Context Menu to select shape

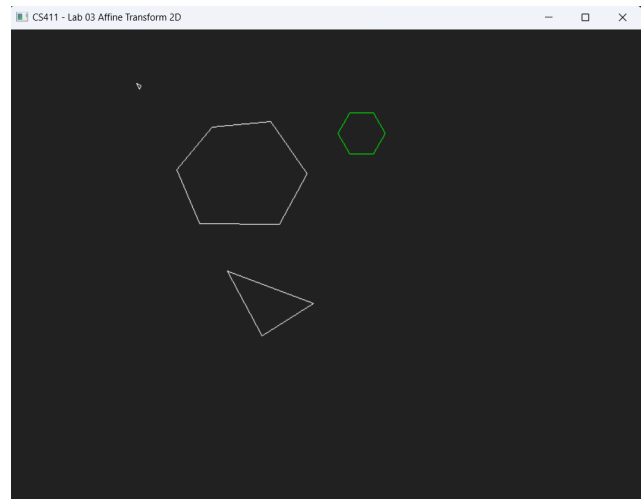


Figure 6: A shape is correctly selected

2.4 Affine Transformations

Once a polygon is securely selected (turns Green) via the context menu, Affine Transform mapping logic opens up targeting that exact shape:

- **Translation (1 pixel per click):** Use the **Arrow Keys** (Up, Down, Left, Right) to relocate the targeted shape linearly.
- **Rotation (1 degree per click):** Use the **L** key to rotate Counter-Clockwise precisely, or the **R** key to rotate Clockwise.
- **Scaling (10% per click):** Use the **+** key to iteratively expand/zoom-in the targeted geometry by 10%, and the **-** key to compress its vector dimensions by 10%.

We handle them in functions called `Keyboard` and `SpecialKeyboard`:

```
void Keyboard(unsigned char key, int x, int y) {
    if (selectedPolygonIndex == -1 || selectedPolygonIndex >= polygons.size())
        return;
    PolygonShape& poly = polygons[selectedPolygonIndex];
    if (key == 'L' || key == 'l') {
        poly.angle += 1.0f; // Rotate counter-clockwise
    } else if (key == 'R' || key == 'r') {
        poly.angle -= 1.0f; // Rotate clockwise
    } else if (key == '+') {
        poly.scale *= 1.1f;
    } else if (key == '-') {
```

```

        poly.scale /= 1.1f;
    }
    glutPostRedisplay();
}

void SpecialKeyboard(int key, int x, int y) {
    if (selectedPolygonIndex == -1 || selectedPolygonIndex >= polygons.size
    ())
        return;
    PolygonShape& poly = polygons[selectedPolygonIndex];
    switch (key) {
        case GLUT_KEY_UP:    poly.ty -= 1.0f; break;
        case GLUT_KEY_DOWN:  poly.ty += 1.0f; break;
        case GLUT_KEY_LEFT:  poly.tx -= 1.0f; break;
        case GLUT_KEY_RIGHT: poly.tx += 1.0f; break;
    }
    glutPostRedisplay();
}

```

3 Technical Implementation & Algorithms

3.1 Ray-Casting Algorithm (Hit Detection)

To effectively identify overlaps, hovering mechanics, and click interceptions, the codebase applies the prominent **Ray-Casting Algorithm** located across `helper.cpp` (`pointInPolygon`).

- It extends an infinite horizontal vector ray strictly to the right of the targeted (px, py) mouse coordinates.
- It iterates across all existing geometry vertices mapping edge line segments.
- Extrapolating odd versus even crossing intersection counts directly identifies inner versus outer boundaries.
- **Reverse Transforms:** When testing hovered polygons during translation (tx, ty) , the coordinates being tested inversely subtract (tx, ty) , bridging the mouse back cleanly against the raw mathematical array.

3.2 OpenGL Matrix Pipeline

Transformations natively map against data tracked individually for every shape's metadata (tx, ty, cx, cy, angle, scale variables inside PolygonShape). When drawn, each sequence wraps independently inside OpenGL fixed-pipeline matrix stacks natively circumventing destructive mathematical overrides:

```
glPushMatrix();
// 1. Translating logic wrapping the entire structural offset block
glTranslatef(polygons[i].tx, polygons[i].ty, 0);

// 2. Modifying base center mappings before acting matrix transforms
glTranslatef(polygons[i].cx, polygons[i].cy, 0);
glRotatef(polygons[i].angle, 0, 0, 1);
glScalef(polygons[i].scale, polygons[i].scale, 1);
glTranslatef(-polygons[i].cx, -polygons[i].cy, 0);

// 3. Render Vertices mapping...
glBegin(GL_LINE_LOOP);
for (size_t j = 0; j < polygons[i].vertices.size(); j++) {
    glVertex2f(polygons[i].vertices[j].x, polygons[i].vertices[j].y);
}
glEnd();
glPopMatrix();
```

3.3 FreeGLUT Menu State Management

A heavy graphical roadblock regarding FreeGLUT is that it locks native event trackers (like glutPassiveMotionFunc) entirely whenever a menu triggers GLUT_MENU_IN_USE. To resolve lingering state desynchronization problems (where right-clicking would open the wrong menu), the architecture leverages glutMenuStatusFunc(MenuStatus).

The codebase waits gracefully for the GLUT_MENU_NOT_IN_USE system evaluation. It then drops the UI lock exactly upon closure, securely triggering a forced hover-update check to rebuild the dynamic menu exactly *before* the next mouse click evaluation, preventing desync locks natively.

4 Conclusion

This implementation directly meets all affine transformation mechanics natively mapped gracefully over stable, scalable object structures. It successfully separates raw mathematical architecture from rendering event interfaces efficiently, providing a seamless interactive computer graphics experience.